Программирование серверных приложений

Конспект написан Заблоцким Данилом

Осенний семестр 2024

Эти записи не одобряются лекторами, и я вношу в них изменения (часто существенно) после лекций. Они далеко не точно отражают то, что на самом деле читалось, и, в частности, все ошибки почти наверняка мои.

Оглавление

1 Многопоточность **3**
1.1 Основные термины 3
1.1.1 Жизенный цикл процесса (общая ситуация) 3

Глава 1

Многопоточность

1.1 Основные термины

Определение 1 (Процесс). *Процесс (process) / задача (task)* – это цепочка ....

Замечание. Процессы выполняются (псевдо) параллельно: каждому выделяются по-очереди квант времени, в течение которого процесс может работать.

1.1.1 Жизненный цикл процесса (общая ситуация)

Состояние процесса:

- новый (процесс только что создан, но еще не встал в очередь);
- готовый к выполнению (находится в очереди);
- активный – выполняющийся процесс;
- завершенный – процесс, который завершился, но часть его структур данных еще существует;
- блокированный процесс – процесс, который не может выполняться по причине ожидания какого-то события или ресурса.

Определение 2 (Поток управления). *Поток управления (thread)* – это цепочка выполняющихся команд в рамках какого-то процесса. Поток использует ресурсы процесса.

Все потоки процесса разделяют ресурсы процессора.

Замечание. Процессу выделяется адресное пространство памяти. Адресное пространство процесса может быть больше физической памяти (виртуальная память).

При переключении процессов необходимо сохранить контекст старого процесса и восстановить контекст нового.

Потоки чередуются, как и процессы. При этом переключение потоков выполняется быстрее, чем у процессов. Также передача данных между потоками быстрее.

Пример.

- интерфейс и сборка проектов в IDE выделяются в отдельные потоки;
- фоновое автосохранение;

◇

Определение 3 (Атомарная операция)**.** *Атомарная операция* – операция, внутри которой невозможно переключение процессов / потоков.

Java позволяет создавать потоки и выполнять их параллельно. Поток в Java коде представлен объектом класса `Thread`.

В выполняющейся программе при старте создается главный поток, все остальные создаются во время работы программы.

Создание потоков:

Способ 1. Создать объект класса `Thread` специальным конструктором по объекту `Runnable`. Интерфейс `Runnable` описывает действия, коиторые должен выполнять поток (задачу).

```
1 public interface Runnable {
2   void run();
3 }
4
```

Способ 2. Расширить класс `Thread`, переопределив его метод `run()`, создать объект конструктором по умолчанию.

После создания объект-потока надо вызвать его метод `start()`.

Пример.

```
public class Demo {
  public static void main(....) {
    S.o.p.("Start");
    Thread thread = new Thread ( () -> {
      for (i = 3; i > 0; i--) {
        S.o.p.("(" + i + ")");
      }
      S.o.p.("Launch");
    }
  );
  thread start();
  S.o.p.("Finish");
}
```

◇

При создании потоку можно назначить имя.

`Thread.currentThread` – текущий поток.

```
public static main(...) {
  S.o.p.("Start");
  Runnable task = () -> {
    String name = Thread.currentThread.getName();
    for (int i = 3; i > 0; i--) {
      S.o.p.(name + "(" + i + ")");
    }
    S.o.p.(name + "launch!");
  };
}
```